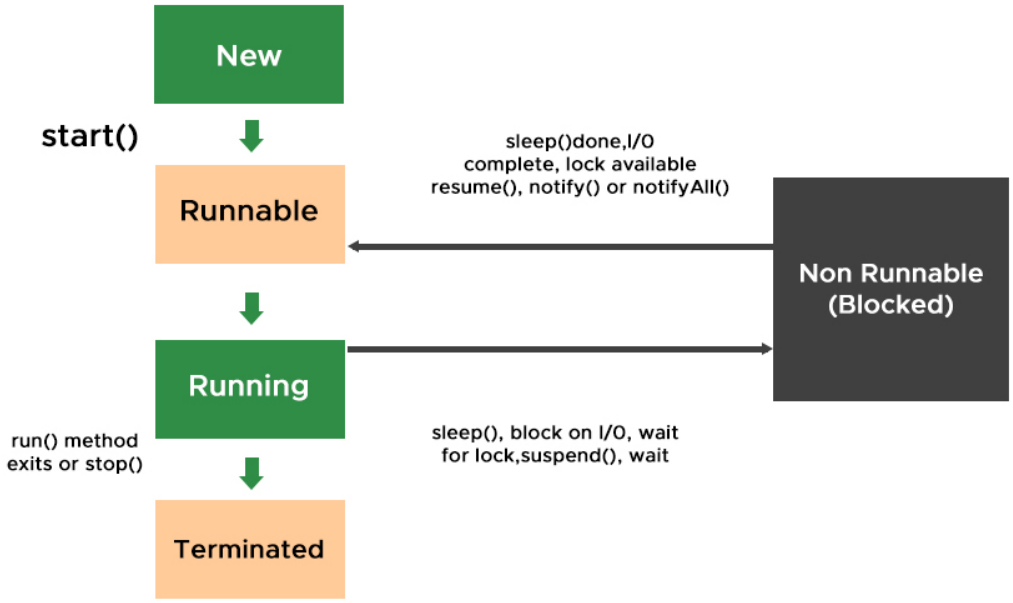
Unit 5: Multithreading, Java Collections 6 hrs.

5.1 Introduction Multithreading
- 5.1.1 Thread class and Runnable Interface
- 5.1.2 Creating Threads
- 5.1.3 Stopping and Blocking a Thread
- 5.1.4 Life Cycle of a Thread
- 5.1.5 Using Thread Methods
- 5.1.6 Thread Priority

5.2 Collection Framework in Java
- 5.2.1 Overview of java collection framework.
- 5.2.2 Commonly used collection classes- ArrayList, HashMap, HashSet, Vector

Java.lang.Thread Class in Java

Java provides a thread class that has various method calls in order to manage the behavior of threads by providing constructors and methods to perform operations on threads.



Creating a thread:

There are two ways(methods) to create a thread in java programming

a) By Extending Thread Class
b) By implementing Runnable interface

Simple program that is created by extending Thread class

```
package thread;

// Creating thread By Extending To Thread class

class MyThread extends Thread {

    // Run() method for our thread
    public void run() {
        // Print statement
        System.out.println("Thread is running created by
extending to parent Thread class");
    }

    // Main driver method
    public static void main(String[] args) {

        // Creating object of our thread class inside main()
        MyThread myThread = new MyThread();

        // Starting the thread
        myThread.start();
    }
}
```

Create a class Odd by extending Thread class which display odd number from 1 to 100 in every 0.4 seconds.

```
public class Odd extends Thread {
    @Override
    public void run() {
        for (int i = 1; i < 100; i += 2) {
            System.out.println("O: " + i);
            try {
                Thread.sleep(400);
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
        }
```

```
    }

    public static void main(String[] args) {
        Odd o = new Odd();
        o.setPriority(Thread.MAX_PRIORITY);
        o.start();
    }
}
```

Runnable interface in Java

java.lang.Runnable is an interface that is to be implemented by a class whose instances are intended to be executed by a thread. There are two ways to start a new Thread – Subclass Thread and implement Runnable. There is no need of subclassing a Thread when a task can be done by overriding only run() method of Runnable.

Steps to create a new thread using Runnable

1. Create a Runnable implementer and implement the run() method.

2. Instantiate the Thread class and pass the implementer to the Thread, Thread has a constructor which accepts Runnable instances.

3. Invoke start() of Thread instance, start internally calls run() of the implementer. Invoking start() creates a new Thread that executes the code written in run(). Calling run() directly doesn't create and start a new Thread, it will run in the same thread. To start a new line of execution, call start() on the thread.

Simple Example to implement Runnable interface

```
public class RunnableDemo implements Runnable {

    public static void main(String[] args) {
        System.out.println("Main thread is- "
                + Thread.currentThread().getName());
        Thread t1 = new Thread(new RunnableDemo());
        t1.start();
    }
```

```
    public void run() {
        System.out.println(Thread.currentThread().getName()
                + ", executing run() method!");
    }
}
```

A program to display even number from 100 to 2 in every 0.7 per seconds using Runnable interface.

```
public class Even implements Runnable{
   public void run(){
       for (int i = 100; i >1; i-=2) {
           System.out.println("E: "+i);
           try {
               Thread.sleep(700);
           }catch (Exception e){
               System.out.println(e.getMessage());
           }
       }
   }

    public static void main(String[] args) {
        Even e = new Even();
        Thread et = new Thread(e);
        et.start();
    }
}
```

Methods of Thread class:

Now let us do discuss all the methods of this class are illustrated as follows:

Methods	Action Performed
activeCount()	Returns an estimate of the number of active threads in the current thread's thread group and its subgroups
checkAccess()	Determines if the currently running thread has permission to modify this thread
clone()	Throws CloneNotSupportedException as a Thread can not be meaningfully cloned
currentThread()	Returns a reference to the currently executing thread object
dumpStack()	Prints a stack trace of the current thread to the standard error stream
enumerate(Thread[] tarray)	Copies into the specified array every active thread in the current thread's thread group and its subgroups
getAllStackTraces()	Returns a map of stack traces for all live threads
getContextClassLoader()	Returns the context Class Loader for this Thread
getDefaultUncaughtExceptionHandler()	Returns the default handler invoked when a thread abruptly terminates due to an uncaught exception
getId()	Returns the identifier of this Thread
getName()	Returns this thread's name

Methods	Action Performed
getPriority()	Returns this thread's priority
getStackTrace()	Returns an array of stack trace elements representing the stack dump of this thread
getState()	Returns the state of this thread
getThreadGroup()	Returns the thread group to which this thread belongs
getUncaughtExceptionHandler()	Returns the handler invoked when this thread abruptly terminates due to an uncaught exception
holdsLock(Object obj)	Returns true if and only if the current thread holds the monitor lock on the specified object
interrupt()	Interrupts this thread
interrupted()	Tests whether the current thread has been interrupted
isAlive()	Tests if this thread is alive
isDaemon()	Tests if this thread is a daemon thread
isInterrupted()	Tests whether this thread has been interrupted
join()	Waits for this thread to die

Methods	Action Performed
join(long millis)	Waits at most millis milliseconds for this thread to die
run()	If this thread was constructed using a separate Runnable run object, then that Runnable object's run method is called; otherwise, this method does nothing and returns
setContextClassLoader(ClassLoader cl)	Sets the context ClassLoader for this Thread
setDaemon(boolean on)	Marks this thread as either a daemon thread or a user thread
setDefaultUncaughtExceptionHandler(Thread.UncaughtExceptionHandler eh)	Set the default handler invoked when a thread abruptly terminates due to an uncaught exception, and no other handler has been defined for that thread
setName(String name)	Changes the name of this thread to be equal to the argument name.
setUncaughtExceptionHandler(Thread.UncaughtExceptionHandler eh)	Set the handler invoked when this thread abruptly terminates due to an uncaught exception
setPriority(int newPriority)	Changes the priority of this thread
sleep(long millis)	Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds, subject to the precision and accuracy of system timers and schedulers

Methods	Action Performed
start()	Causes this thread to begin execution; the Java Virtual Machine calls the run method of this thread
toString()	Returns a string representation of this thread, including the thread's name, priority, and thread group
yield()	A hint to the scheduler that the current thread is willing to yield its current use of a processor

Java Thread Priority

As we already know java being completely object-oriented works within a multithreading environment in which thread scheduler assigns the processor to a thread based on the priority of thread. Whenever we create a thread in Java, it always has some priority assigned to it. Priority can either be given by JVM while creating the thread or it can be given by the programmer explicitly.

Priorities in threads is a concept where each thread is having a priority which in layman's language one can say every object is having priority here which is represented by numbers ranging from 1 to 10.

- The default priority is set to 5 as excepted.
- Minimum priority is set to 1.
- Maximum priority is set to 10.

Here 3 constants are defined in it namely as follows:

- public static int NORM_PRIORITY
- public static int MIN_PRIORITY
- public static int MAX_PRIORITY

The accepted value of priority for a thread is in the range of 1 to 10.

To get and set priority of a thread in java

a) public final int getPriority(): java.lang.Thread.getPriority() method returns priority of given thread.
b) public final void setPriority(int newPriority): java.lang.Thread.setPriority() method changes the priority of thread to the value newPriority. This method throws IllegalArgumentException if value of parameter newPriority goes beyond minimum(1) and maximum(10) limit.

For example,

```
// Java Program to Illustrate Priorities in Multithreading  via help of getPriority() and
setPriority() method

class ThreadDemo extends Thread {

   public void run() {
      System.out.println("Inside run method");
   }

   public static void main(String[] args) {
      // Creating random threads with the help of above class
      ThreadDemo t1 = new ThreadDemo();
      ThreadDemo t2 = new ThreadDemo();
      ThreadDemo t3 = new ThreadDemo();

      // Thread 1
      // Display the priority of above thread using getPriority() method
      System.out.println("t1 thread priority : " + t1.getPriority());

      // Thread 1
      // Display the priority of above thread
      System.out.println("t2 thread priority : " + t2.getPriority());

      // Thread 3
      System.out.println("t3 thread priority : " + t3.getPriority());

      // Setting priorities of above threads by passing integer arguments
```

```
        t1.setPriority(2);
        t2.setPriority(5);
        t3.setPriority(8);

        // t3.setPriority(21); will throw IllegalArgumentException

        // 2
        System.out.println("t1 thread priority : " + t1.getPriority());

        // 5
        System.out.println("t2 thread priority : " + t2.getPriority());

        // 8
        System.out.println("t3 thread priority : " + t3.getPriority());

        // Displays the name of  currently executing Thread
        System.out.println("Currently Executing Thread : " + Thread.currentThread().getName());

        System.out.println("Main thread priority : " + Thread.currentThread().getPriority());

        // Main thread priority is set to 10
        Thread.currentThread().setPriority(10);

        System.out.println("Main thread priority : " + Thread.currentThread().getPriority());
    }
}
```

Java Collections

Any group of individual objects that are represented as a single unit is known as a Java Collection of Objects. In Java, a separate framework named the “Collection Framework” has been defined in JDK 1.2 which holds all the Java Collection Classes and Interface in it.

In Java, the Collection interface (java.util.Collection) and Map interface (java.util.Map) are the two main “root” interfaces of Java collection classes.

A framework is a set of classes and interfaces which provide a ready-made architecture.

Before the Collection Framework (or before JDK 1.2) was introduced, the standard methods for grouping Java objects (or collections) were Arrays or Vectors, or Hashtables. All of these collections had no common

interface. Therefore, though the main aim of all the collections is the same, the implementation of all these collections was defined independently and had no correlation among them. And also, it is very difficult for the users to remember all the different methods, syntax, and constructors present in every collection class.

```java
// Java program to demonstrate why collection framework was needed
import java.util.Hashtable;
import java.util.Vector;

class CollectionDemo {

	public static void main(String[] args)
	{
		// Creating instances of the array, vector and hashtable
		int arr[] = new int[] { 1, 2, 3, 4 };
		Vector<Integer> v = new Vector();
		Hashtable<Integer, String> h = new Hashtable();

		// Adding the elements into the vector
		v.addElement(1);
		v.addElement(2);

		// Adding the element into the hashtable
		h.put(1, "nast");
		h.put(2, "dhangadhi");

		// Array instance creation requires [],
		// while Vector and hastable require ()
		// Vector element insertion requires addElement(),
		// but hashtable element insertion requires put()

		// Accessing the first element of the array, vector and hashtable
		System.out.println(arr[0]);
		System.out.println(v.elementAt(0));
		System.out.println(h.get(1));

		// Array elements are accessed using [],
		// vector elements using elementAt()
		// and hashtable elements using get()
	}
}
```

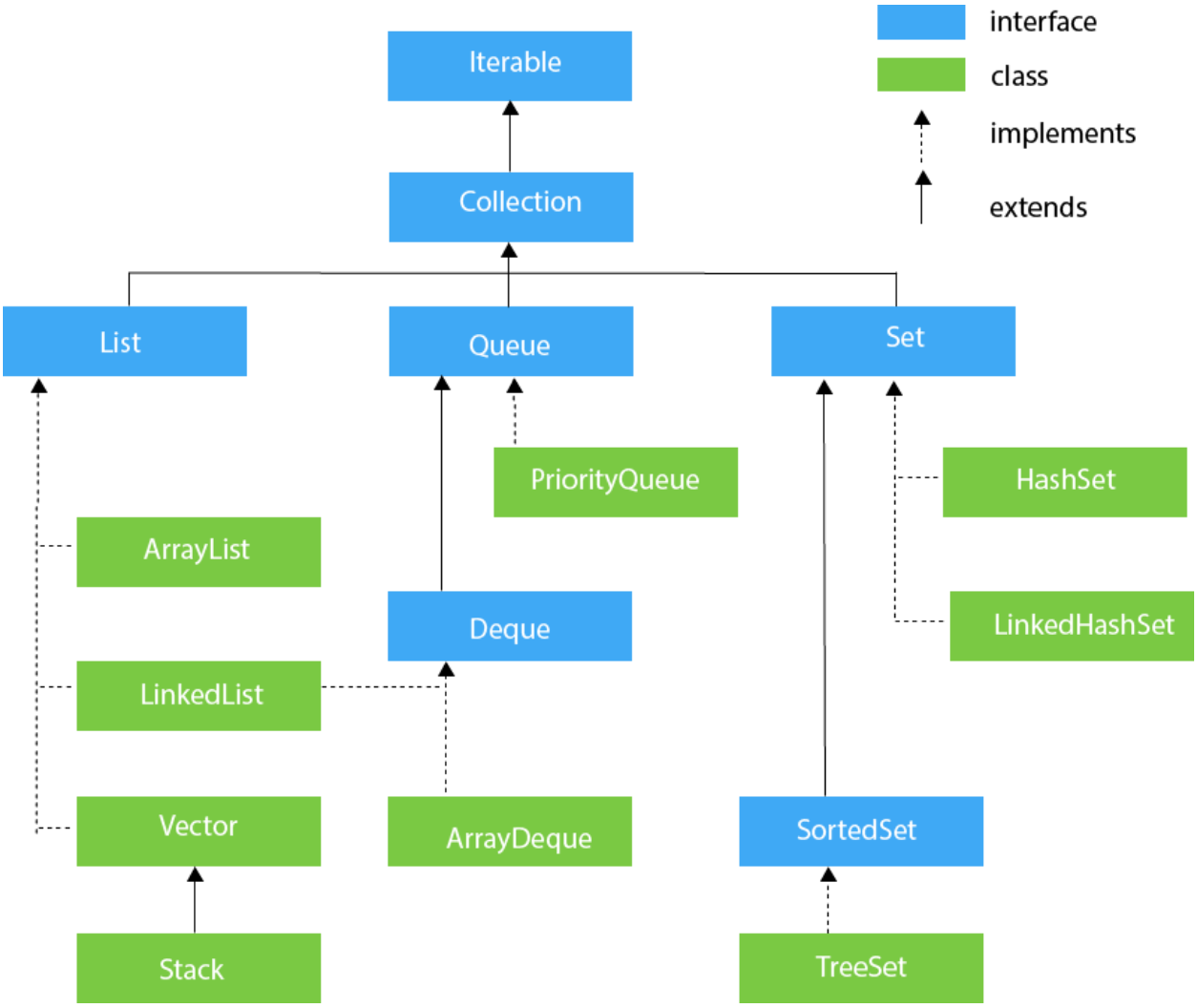
As we can observe, none of these collections (Array, Vector, or Hashtable) implements a standard member access interface, it was very difficult for programmers to write algorithms that can work for all kinds of Collections. Another drawback is that most of the 'Vector' methods are final, meaning we cannot extend the 'Vector' class to implement a similar kind of Collection. Therefore, Java developers decided to come up with a common interface to deal with the above-mentioned problems and introduced the Collection Framework in JDK 1.2 post which both, legacy Vectors and Hashtables were modified to conform to the Collection Framework.

Collection Framework

The Collection in Java is a framework that provides an architecture to store and manipulate the group of objects. Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.

Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

Let us see the hierarchy of Collection framework. The java.util package contains all the classes and interfaces for the Collection framework.



Methods of Collection interface

add(Object)

addAll(Collection c)
clear()
contains(Object o)
containsAll(Collection c)
equals(Object o)
isEmpty()
iterator()
max()
remove(Object o)
removeAll(Collection c)
size()
toArray()

List Interface

For example:
List <T> al = new ArrayList<> ();
List <T> ll = new LinkedList<> ();
List <T> v = new Vector<> ();
Where T is the type of the object

ArrayList

```
// Java program to demonstrate the working of ArrayList
import java.util.ArrayList;

class ArrayListDemo {

	// Main Method
	public static void main(String[] args)
	{

		// Declaring the ArrayList with initial size n
		ArrayList<Integer> al = new ArrayList<Integer>();

		// Appending new elements at the end of the list
		for (int i = 1; i <= 5; i++)
			al.add(i);

		// Printing elements
		System.out.println(al);

		// Remove element at index 3
		al.remove(3);

		// Displaying the ArrayList after deletion
		System.out.println(al);

		// Printing elements one by one
		for (int i = 0; i < al.size(); i++)
```

```
                    System.out.print(al.get(i) + " ");
        }
}
```

Another example to explore some more methods of ArrayList class

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class ArrayListDemo {

    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        //size()
        System.out.println(names.size());


        //add()
        names.add("Motiram");
        System.out.println(names);
        System.out.println(names.size());


        //add()
        names.add("Santosh");
        System.out.println(names);
        System.out.println(names.size());

        //add()
        names.add("Pawan");
        System.out.println(names);
        System.out.println(names.size());

        //add()
        names.add("Nabin");
        System.out.println(names);
        System.out.println(names.size());


        //addFirst()
        names.addFirst("Sumitra");
        System.out.println(names);
        System.out.println(names.size());

        //addFirst()
        names.addLast("Sunil");
        System.out.println(names);
        System.out.println(names.size());

        //add(index, item)
```

```java
names.add(2, "Kiku");
System.out.println(names);
System.out.println(names.size());

//remove()
names.remove(3);
System.out.println(names);
System.out.println(names.size());

//add(index,element)
names.add(2, "Santosh");
System.out.println(names);
System.out.println(names.size());

List<String> anotherNames = new
             ArrayList<>(Arrays.asList("Lokendra",
             "Narendra", "Pradip", "Pravesh"));
System.out.println(anotherNames.size());

//addAll(index,collection)
names.addAll(3, anotherNames);
System.out.println(names);
System.out.println(names.size());

//adding duplicate element
names.add("Nabin");
names.add("Kiku");
for (String n : names) {
    System.out.println(n);
}

//removes all item
// names.clear();
System.out.println(names.size());

//contains()
if(names.contains("Nepal")){
    System.out.println("Found");
}else{
    System.out.println("Not Found");
}

//get(index)
System.out.println(names.get(5));
System.out.println(names);
System.out.println(names.indexOf("Nabin"));

//containsAll(collection)
System.out.println(names.containsAll(
             Arrays.asList("Nabin","Jaigadi")));

//lastIndexOf(item)
```

```
        System.out.println(names.lastIndexOf("Nabin"));

        System.out.println(names.reversed());


        //retainAll()
        System.out.println(names.retainAll(
                    Arrays.asList("Sunil","Nabin")));
        System.out.println(names);
    }
}
```

Set Interface

For example:

Set<T> hs = new HashSet<> ();

Set<T> lhs = new LinkedHashSet<> ();

Set<T> ts = new TreeSet<> ();

Where T is the type of the object.

```
// Java program to demonstrate the working of a HashSet
import java.util.HashSet;
import java.util.Iterator;

public class HashSetDemo {

      // Main Method
      public static void main(String args[])
      {
            // Creating HashSet and adding elements
            HashSet<String> hs = new HashSet<String>();

            hs.add("NAST");
            hs.add("For");
            hs.add("BE Computer");
            hs.add("Is");
            hs.add("Very helpful");

            // Traversing elements
            Iterator<String> itr = hs.iterator();
            while (itr.hasNext()) {
                  System.out.println(itr.next());
            }
      }
}
```

One more example on exploring different methods on HashSet

```
import java.util.*;

public class HashSetDemo {
```

```java
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();

        //size()
        System.out.println(set.size());


        //add()
        for (int i = 0; i < 10; i++) {
            set.add("Item " + i);
        }
        //remove()
        set.remove(3);
        System.out.println(set);
        System.out.println(set.size());


        //adding duplicate element but it do not adds
        set.add("Nabin");
        set.add("Kiku");
        for (String n : set) {
            System.out.println(n);
        }

        //removes all item
        // names.clear();
        System.out.println(set.size());

        //contains()
        if (set.contains("Nepal")) {
            System.out.println("Found");
        } else {
            System.out.println("Not Found");
        }


        //containsAll(collection)

System.out.println(set.containsAll(Arrays.asList("Nabin",
"Jaigadi")));

        //retainAll()
        System.out.println(
             set.retainAll(Arrays.asList("Sunil", "Nabin")));
        System.out.println(set);
```

```
    }
}
```

Map Interface

For example:

Map<T> hm = new HashMap<> ();

Map<T> tm = new TreeMap<> ();

Where T is the type of the object.

HashMap

```
// Java program to demonstrate the working of a HashMap
import java.util.*;

public class HashMapDemo {

    // Main Method
    public static void main(String args[])
    {
        // Creating HashMap and adding elements
        HashMap<Integer, String> hm
        = new HashMap<Integer, String>();

        hm.put(1, "NAST");
        hm.put(2, "For");
        hm.put(3, "BE Computer");

        // Finding the value for a key
        System.out.println("Value for 1 is " + hm.get(1));

        // Traversing through the HashMap
        for (Map.Entry<Integer, String> e : hm.entrySet())
            System.out.println(e.getKey() + " "
                        + e.getValue());
    }
}
```

Another example,

```
import java.util.HashMap;
import java.util.Map;

public class MapDemo {
    public static void main(String[] args) {
        Map<Long, String> contacts = new HashMap<>();

        contacts.put(9858487100L, "NAST Office");
        contacts.put(9858487101L, "NAST Principal");
        contacts.put(98584871010L, "NAST Principal");
        contacts.put(9858487102L, "NAST Computer HOD");
```

```
        contacts.put(9858487103L, "NAST BCA HOD");
        contacts.put(9858487104L, "NAST BBA HOD");
        contacts.put(9858487105L, "NAST BE Civil HOD");
        contacts.put(9858487106L, "NAST");

        //get(index)
        System.out.println(contacts.get(9858487100L));

        //containsKey()
        System.out.println(contacts.containsKey(985848710L));
        //looping through all elements
        contacts.forEach((k, v) -> {
            System.out.print("Mobile: " + k);
            System.out.println("Name: " + v);
        });

        System.out.println(contacts);
    }
}
```

Vector Class in Java

The Vector class implements a growable array of objects. Vectors fall in legacy classes, but now it is fully compatible with collections. It is found in java.util package and implement the List interface.

- Thread-Safe: All methods are synchronized, making it suitable for multi-threaded environments. However, this can lead to performance overhead in single-threaded scenarios.
- Allows Nulls: Can store null elements.
- Enumeration Support: Provides backward compatibility with Enumeration, a legacy way of iterating over elements.

Constructors

1. Vector(): Creates a default vector of the initial capacity is 10.

Vector<E> v = new Vector<E>();

2. Vector(int size): Creates a vector whose initial capacity is specified by size.

Vector<E> v = new Vector<E>(int size);

3. Vector(int size, int incr): Creates a vector whose initial capacity is specified by size and increment is specified by incr. It specifies the number of elements to allocate each time a vector is resized upward.

Vector<E> v = new Vector<E>(int size, int incr);

4. Vector(Collection c): Creates a vector that contains the elements of collection c.

Vector<E> v = new Vector<E>(Collection c);

For example,

```
import java.util.*;
public class VectorDemo {
    public static void main(String[] args) {
        Vector<String> vector = new Vector<>();

        //size()
        System.out.println(vector.size());

        //capacity 10
        System.out.println(vector.capacity());

        //addElement()
        vector.addElement("By Element Method");
        for(int i=0;i<10;i++){
            vector.add("Item "+i);
        }
        //now new capacity becomes 20
        System.out.println("Capacity: "+vector.capacity());

        //clone()
        Vector<String> clonVector=
                         (Vector<String>) vector.clone();

        //display all the items using forEach()
        vector.forEach((e)-> System.out.println(e));

        //replace an item
        clonVector.add(0,"Element 1");
        clonVector.forEach((i)-> System.out.println(i));

        //add()
        vector.add("Motiram");
        System.out.println(vector);
        System.out.println(vector.size());


        //add()
```

```java
vector.add("Santosh");
System.out.println(vector);
System.out.println(vector.size());

//add()
vector.add("Pawan");
System.out.println(vector);
System.out.println(vector.size());

//add()
vector.add("Nabin");
System.out.println(vector);
System.out.println(vector.size());

//addFirst()
vector.addFirst("Sumitra");
System.out.println(vector);
System.out.println(vector.size());

//addFirst()
vector.addLast("Sunil");
System.out.println(vector);
System.out.println(vector.size());

//add(index, item)
vector.add(2, "Kiku");
System.out.println(vector);
System.out.println(vector.size());

//remove()
vector.remove(3);
System.out.println(vector);
System.out.println(vector.size());

//add(index,element)
vector.add(2, "Santosh");
System.out.println(vector);
System.out.println(vector.size());

List<String> anotherNames = new ArrayList<>(
     Arrays.asList("Lokendra", "Narendra", "Pradip"));
System.out.println(anotherNames.size());

//addAll(index,collection)
vector.addAll(3, anotherNames);
System.out.println(vector);
System.out.println(vector.size());
```

```java
        //adding duplicate element
        vector.add("Nabin");
        vector.add("Kiku");
        for (String n : vector) {
            System.out.println(n);
        }

        //removes all item
        // names.clear();
        System.out.println(vector.size());

        //contains()
        if(vector.contains("Nepal")){
            System.out.println("Found");
        }else{
            System.out.println("Not Found");
        }

        //get(index)
        System.out.println(vector.get(5));
        System.out.println(vector);
        System.out.println(vector.indexOf("Nabin"));

        //containsAll(collection)
        System.out.println(vector.containsAll(
                   Arrays.asList("Nabin","Jaigadi")));

        //lastIndexOf(item)
        System.out.println(vector.lastIndexOf("Nabin"));

        System.out.println(vector.reversed());

        Collections.sort(vector);
        System.out.println("Sorted: "+vector);

      vector.sort(Collections.reverseOrder());
        System.out.println("Reverse Order: "+vector);

        //retainAll()
        System.out.println(vector.retainAll(
             Arrays.asList("Sunil","Nabin")));
        System.out.println(vector);
    }
}
```